

AI Chat History

Lab 4 — VibeCoding | Memory Color Pattern Repair Lab

Purpose: Record the AI-assisted discussion and workflow used to understand, debug, improve, test, and prepare the Memory Color Pattern game for submission.

Project	Memory Color Pattern Repair Lab (Simon-style Pygame game)
Language / library	Python / Pygame
Required tasks	1. Fix sequence duplication; 2. Dynamic playback acceleration; 3. Color-specific audio; 4. Player-turn countdown timer
Submission	Before video, after video, updated code, exported AI chat history; separate commits for each task; no PR to SETAPESU26.

Note about this document

This is a compiled, assignment-focused record of the Lab 4 discussion, organized into the questions asked and guidance provided. It is not a raw account-data export or a verbatim export of every message. If your instructor requires the platform's complete original chat, also use ChatGPT's Share or Data Export options and include the share link when available.

Conversation Record

User: Explain the lab task and deliverables.

Assistant: Run the assigned Python game, record a 10-second video before changes, fix the deliberate bug, add the README features one by one, record a 10-second after video, push updated code to a personal repository, keep separate commits for each task, and export the AI chat history. Do not raise a pull request to SETAPESU26.

User: Shared the README and Pygame installation errors.

Assistant: ``pip install requirments.txt`` was incorrect. If a requirements file exists, use ``python -m pip install -r requirements.txt``. The error log showed Python 3.14 trying to build Pygame from source and failing. The suggested approach was to use a supported installed Python version in a virtual environment, install Pygame, and verify the import.

User: Python 3.12 was not found; Python 3.11 and 3.14 were installed.

Assistant: Use Python 3.11: check with ``py -3.11 --version``, create a virtual environment with ``py -3.11 -m venv .venv``, activate it, install Pygame, and run ``python main.py``. If activation is blocked, call ``.venv\Scripts\python.exe`` directly.

User: Asked what the game is and what needs to be done.

Assistant: It is a Simon-style memory game. The computer flashes colored pads and the player clicks the same colors in the same order. Correct play advances to a longer sequence; wrong input triggers Game Over; R restarts. The tasks are sequence bug repair, playback acceleration, sound effects, and a player-turn countdown timer.

User: Asked what was wrong in the project.

Assistant: The key deliberate bug in ``game/game_engine.py`` is ``self.sequence += self.sequence + [new_color]``, which duplicates the existing pattern. Replace it with ``self.sequence.append(new_color)``. The other tasks require round-dependent flash/pause timing, distinct tones for four colors, and a countdown bar that triggers Game Over on timeout.

User: Asked how to complete Task 1 and which files to change.

Assistant: Task 1 only requires changing ``game/game_engine.py`` inside ``start_next_round()``. Replace the faulty concatenation with ``self.sequence.append(new_color)``, save, run a syntax check, play multiple rounds, test wrong input and restart, then commit separately. ``main.py`` and ``game/color_button.py`` normally do not need changes for Task 1.

User: Confirmed the workflow should be one task, one commit and push at a time.

Assistant: Complete and test Task 1, commit and push it; then do the same for Task 2, Task 3, and Task 4. Verify ``git remote -v`` points to the personal fork. Do not push directly to SETAPESU26 or open a PR to it.

User: Asked which files change for each task.

Assistant: Task 1 — ``game/game_engine.py``; Task 2 — ``game/game_engine.py``; Task 3 — usually ``game/game_engine.py``, with ``main.py`` only if audio initialization needs adjustment; Task 4 — ``game/game_engine.py``. ``game/color_button.py`` normally remains unchanged unless visual changes are needed.

User: Said the repo was forked, original game run, and before video recorded.

Assistant: Next, fix and test Task 1, commit and push; then implement and commit Tasks 2, 3, and 4 separately. After testing, record the after video, export chat history, and upload the deliverables under ``Lab-4``.

User: Asked what “Do not raise PR to the main repo (SETAPESU26)” means.

Assistant: A fork is your own copy of the instructor's repository. A pull request asks the original repository owner to review and potentially merge changes. Push to your personal fork only; do not open a PR to SETAPESU26.

User: Asked how to create ``chat-history.pdf``.

Assistant: Use browser Print → Save as PDF for a conversation, or ChatGPT Settings → Data Controls → Export Data for a broader export. Check that the PDF includes the relevant prompts and responses. If available, create a share link

and include it.

User: Asked how to play the game.

Assistant: Run ``python main.py``; watch the sequence, then left-click the same colors in order. A correct sequence advances to a longer sequence. A wrong click ends the game; timer expiration also ends it if implemented. Press R on Game Over to restart.

User: Asked to build a chat history.

Assistant: This compiled assignment-focused chat record was prepared. Supplement it with ChatGPT's original export/share link if a verbatim complete history is required.

Implementation Plan and Technical Notes

Task 1 — Fix sequence duplication

Problem: ``self.sequence += self.sequence + [new_color]`` repeats the current sequence. Fix: use ``self.sequence.append(new_color)`` so exactly one color is added. Test multiple rounds and verify sequence length increases by one each time.

Task 2 — Dynamic playback acceleration

Reduce flash and pause durations as the round increases, with minimum limits. Preserve the non-blocking, time-based playback state machine and verify that no steps are skipped.

Task 3 — Sound effects

Assign a distinct pitch to Red, Blue, Green, and Yellow. Play the corresponding tone during automated playback and on player clicks. Avoid triggering sounds repeatedly from a per-frame rendering method.

Task 4 — Player-turn countdown timer

Start the countdown when WATCH ends and PLAYER_TURN begins. Draw a visible timer bar, update it without blocking the game loop, trigger GAME_OVER on timeout, and reset it at the next player turn and on restart.

Useful verification commands

```
python -m py_compile main.py game\game_engine.py game\color_button.py
python main.py
git status
git diff
git log --oneline
git remote -v
git push
```

Example commit messages

- Fix sequence duplication bug
- Add dynamic playback acceleration
- Add color-specific sound effects
- Add player turn countdown timer

Final submission checklist

- 10-second before video (before.mp4)
- 10-second after video (after.mp4)
- Updated source code pushed to personal GitHub fork
- Separate commits for each task
- Complete AI chat history PDF/document and share link if available
- Deliverables uploaded under Lab-4 in the lab repository
- No pull request opened against SETAPESU26